

MANUAL TÉCNICO OPINIONES FIUSAC

Plataforma web para opiniones sobre cursos y catedráticos

Fecha: 16 de septiembre de 2026

1. Objetivo del manual

Este manual reúne la información técnica necesaria para comprender, instalar y dar mantenimiento al proyecto Opiniones FIUSAC. Incluye la arquitectura, los componentes principales, el modelo de datos, la API, el proceso de autenticación y las funciones disponibles en el frontend.

2. Descripción general del sistema

Opiniones FIUSAC es una aplicación web dirigida a estudiantes. Desde la plataforma se puede crear una cuenta, iniciar sesión, consultar o publicar opiniones sobre cursos y catedráticos, comentar publicaciones, buscar perfiles por registro académico y llevar un control personal de los cursos aprobados.

3. Arquitectura del sistema

La solución utiliza una arquitectura cliente-servidor de tres capas:

- Frontend: aplicación React ejecutada en el navegador.
- Backend: API REST desarrollada con Node.js y Express.
- Persistencia: base de datos MySQL.

El frontend envía solicitudes HTTP a la API. El backend valida cada petición, consulta o actualiza la información en MySQL y responde en formato JSON. Para acceder a las operaciones protegidas se utiliza un token JWT.

4. Tecnologías utilizadas

Componente	Tecnología	Función
Frontend	React + React Router	Interfaz y navegación
Backend	Node.js + Express	API REST y lógica del servidor
Base de datos	MySQL	Persistencia de usuarios, cursos, publicaciones y comentarios
Autenticación	JWT	Control de acceso a endpoints protegidos
Contraseñas	bcryptjs	Hash y verificación de contraseñas
Configuración	dotenv	Carga de variables de entorno
Comunicación	HTTP/JSON	Intercambio de información frontend-backend

5. Estructura lógica del backend

El servidor principal configura Express, CORS y el procesamiento de JSON. La API se publica bajo el prefijo /api y, de forma predeterminada, se ejecuta en el puerto 3000.

Archivos principales:

- server.js: configuración de Express y definición de endpoints.
- db.js: conexión/pool de MySQL y comprobación de conexión.
- auth.js: generación y validación de tokens JWT.

server.js utiliza pool y checkDatabase desde db.js, además de createToken y requireAuth desde auth.js.

6. Base de datos

El archivo schema.sql crea la base de datos pi_desarrollo_web y define seis tablas principales:

- users: información de los estudiantes y hash de contraseña.
- courses: catálogo de cursos, código, nombre, sección y créditos.
- teachers: información de los catedráticos y su asociación con los cursos.
- posts: publicaciones realizadas por usuarios sobre cursos o catedráticos.
- comments: comentarios asociados a publicaciones y usuarios.
- user_courses: relación muchos-a-muchos entre usuarios y cursos aprobados.

7. Relaciones principales

- users 1:N posts: un usuario puede crear muchas publicaciones.
- users 1:N comments: un usuario puede crear muchos comentarios.
- posts 1:N comments: una publicación puede tener muchos comentarios.
- courses 1:N teachers: un curso puede estar asociado con varios catedráticos.
- courses 1:N posts: una publicación puede referirse a un curso.
- teachers 1:N posts: una publicación puede referirse a un catedrático.
- users N:M courses mediante user_courses: un estudiante puede registrar varios cursos y un curso puede aparecer en varios perfiles.

8. Regla de integridad de publicaciones

La columna subject_type indica si una publicación corresponde a un curso o a un catedrático. Cuando su valor es course, se guarda course_id y teacher_id permanece en NULL; cuando es teacher, se guarda teacher_id y course_id permanece en NULL. Una restricción CHECK en el esquema SQL evita que ambos campos se utilicen al mismo tiempo.

9. Autenticación y autorización

El registro solicita registro académico, nombres, apellidos, correo y contraseña. La contraseña se transforma en un hash con bcryptjs antes de almacenarse. El backend exige una contraseña de al menos seis caracteres en el registro y en el cambio de contraseña.

Al iniciar sesión, el servidor verifica el registro académico y la contraseña. Si son válidos, genera un JWT con el identificador del usuario y su registro académico, con una duración de ocho horas.

Los endpoints protegidos requieren el encabezado Authorization con formato Bearer <token>. El frontend conserva el token y los datos básicos del usuario en localStorage, y añade el token automáticamente a cada solicitud protegida.

10. Endpoints de la API

Método	Endpoint	Protegido	Descripción
GET	/api/health	No	Comprueba conexión con MySQL.
POST	/api/auth/register	No	Registra un usuario.
POST	/api/auth/login	No	Autentica al usuario y devuelve JWT.
POST	/api/auth/reset-password	No	Actualiza contraseña verificando registro y correo.
GET	/api/catalog/courses	No	Obtiene catálogo de cursos.
GET	/api/catalog/teachers	No	Obtiene catálogo de catedráticos.
GET	/api/posts	Sí	Lista publicaciones y permite filtros.
POST	/api/posts	Sí	Crea una publicación.
GET	/api/posts/:id/comments	Sí	Obtiene comentarios de una publicación.
POST	/api/posts/:id/comments	Sí	Agrega un comentario.
GET	/api/users/me	Sí	Obtiene perfil del usuario autenticado.
PUT	/api/users/me	Sí	Actualiza nombres, apellidos y correo.
GET	/api/users/:academicRecord	Sí	Consulta perfil por registro académico.
GET	/api/users/:academicRecord/courses	Sí	Consulta cursos aprobados y créditos.
POST	/api/users/me/courses	Sí	Agrega un curso aprobado.
DELETE	/api/users/me/courses/:courseId	Sí	Elimina un curso aprobado.

11. Filtros de publicaciones

GET /api/posts admite los parámetros courseId, teacherId, courseName y teacherName. Es posible combinar varios filtros en una misma consulta. Los resultados se ordenan de la publicación más reciente a la más antigua, y los nombres de cursos y catedráticos se obtienen mediante JOIN.

12. Frontend

La aplicación React contiene las siguientes áreas funcionales:

- Inicio de sesión.
- Registro de usuarios.
- Recuperación/cambio de contraseña.
- Inicio con listado y filtros de publicaciones.
- Creación de publicaciones sobre cursos o catedráticos.
- Visualización y creación de comentarios.
- Perfil propio y perfiles de otros estudiantes.
- Administración de cursos aprobados y cálculo de créditos.
- Búsqueda de perfiles mediante registro académico.

13. Rutas del frontend

Ruta	Función
/login	Inicio de sesión
/registro	Creación de cuenta
/recuperar	Cambio de contraseña
/	Listado principal de publicaciones
/publicar	Crear publicación
/perfil	Perfil del usuario autenticado
/perfil:academicRecord	Perfil de estudiante por registro académico

14. Control de acceso en React

PublicOnly impide que un usuario con sesión activa vuelva a las pantallas de inicio de sesión, registro o recuperación. PrivateOnly protege las vistas privadas y redirige a /login cuando no hay una sesión válida. Los datos de la sesión se conservan en localStorage.

15. Flujo principal de operación

1. El usuario abre el frontend.
2. Si no existe sesión, puede registrarse o iniciar sesión.
3. El frontend envía las credenciales al endpoint de autenticación.
4. El backend consulta MySQL y valida la contraseña mediante bcryptjs.
5. El backend devuelve un JWT.
6. El frontend guarda token y datos básicos del usuario.
7. En las operaciones protegidas, el frontend envía el encabezado Authorization con el formato Bearer <token>.
8. El backend valida el JWT y ejecuta la operación solicitada.
9. Los datos se leen o escriben en MySQL y la API devuelve JSON.

16. Instalación y configuración

El procedimiento general de instalación es el siguiente. Antes de ejecutarlo, conviene revisar los scripts disponibles en los archivos package.json del frontend y del backend.

1. Instalar Node.js y npm.
2. Instalar MySQL Server.
3. Crear la base de datos ejecutando schema.sql en MySQL.
4. Configurar las variables de entorno del backend para conexión a MySQL, puerto, URL del frontend y JWT_SECRET.
5. Instalar las dependencias del backend con npm install.
6. Instalar las dependencias del frontend con npm install.
7. Iniciar primero MySQL y luego el backend.
8. Iniciar el frontend y verificar que VITE_API_URL apunte a la API.

17. Variables de configuración

Variable	Uso	Valor predeterminado
PORT	Puerto del backend	3000
FRONTEND_URL	Origen permitido por CORS	http://localhost:5173
JWT_SECRET	Clave para firmar JWT	Clave de desarrollo incluida en código; debe cambiarse
DB_HOST	Host MySQL	localhost
DB_PORT	Puerto MySQL	3306
DB_USER	Usuario MySQL	root
DB_PASSWORD	Contraseña MySQL	vacía
DB_NAME	Base de datos	pi_desarrollo_web
DB_CHARSET	Codificación	utf8mb4

18. Pruebas técnicas básicas

- Verificar GET /api/health y confirmar que devuelve database: connected.
- Registrar un usuario con datos válidos.
- Intentar registrar el mismo registro académico o correo y comprobar el error de duplicado.
- Iniciar sesión con credenciales válidas e inválidas.
- Crear una publicación sobre un curso y otra sobre un catedrático.
- Consultar comentarios y agregar un comentario.
- Aplicar filtros por identificador y nombre de curso o catedrático.
- Editar el perfil y comprobar la actualización.
- Agregar y eliminar un curso aprobado y comprobar el cálculo de créditos.
- Intentar acceder a rutas protegidas sin JWT y verificar una respuesta 401.

19. Manejo de errores

La API utiliza códigos HTTP para indicar el resultado de cada solicitud: 400 para datos incompletos o inválidos, 401 cuando falta autenticación o las credenciales son incorrectas, 404 para recursos inexistentes, 409 para registros duplicados y 500 para errores internos.

También existe un manejador global que registra el detalle del error en la consola del servidor y devuelve al cliente un mensaje general, sin exponer información interna.

20. Consideraciones de seguridad

- No utilizar en producción la clave JWT de desarrollo definida como valor alternativo.
- Definir secretos y credenciales mediante variables de entorno.
- Utilizar HTTPS en despliegues reales.
- Revisar la estrategia de recuperación de contraseña antes de producción; actualmente se valida mediante registro académico y correo.

- Aplicar una política de contraseñas más sólida y limitar la cantidad de solicitudes por cliente (rate limiting).
- No almacenar contraseñas en texto plano; el diseño actual utiliza bcryptjs.

21. Mantenimiento y ampliación

Al incorporar nuevas funciones conviene mantener la separación entre frontend, API y base de datos. Si la función necesita guardar información, primero se ajusta el esquema; después se implementa el endpoint correspondiente y, por último, se integra la interfaz en React. Así se reduce el acoplamiento y resulta más sencillo probar cada capa.

22. Archivos que conviene documentar en el repositorio

- README.md con instalación, scripts y requisitos.
- .env.example con nombres de variables, sin secretos reales.
- schema.sql con estructura y datos iniciales.
- server.js con endpoints.
- db.js con configuración de conexión.
- auth.js con JWT.
- api.js con cliente HTTP del frontend.
- App.jsx y hojas de estilo del frontend.

23. Alcance del manual

Este manual describe la estructura y el comportamiento observados en los archivos disponibles del proyecto. Antes de realizar un despliegue en producción se deben confirmar los scripts definidos en package.json, las variables de entorno, la configuración de la base de datos, las reglas de CORS y los valores utilizados por los módulos de autenticación y acceso a datos.

24. Resumen técnico

Opiniones FIUSAC utiliza React en el cliente, Node.js con Express en el servidor y MySQL para almacenar la información. La autenticación se gestiona con JWT y las contraseñas se protegen con bcryptjs. El modelo relacional mantiene separados los usuarios, cursos, catedráticos, publicaciones, comentarios y cursos aprobados. El frontend consume los endpoints REST por medio de un cliente HTTP centralizado.